

Temple App AI Architecture Guide

RAG pipeline, agentic UI, service design, implementation steps, and learning material

Prepared for	Temple App product and engineering team
Primary target	Temple-scoped AI assistant for devotees and temple admins
Recommended stack	OpenAI Responses API + GPT-4.1 + text-embedding-3-small + pgvector

Recommended starting point

Build a new temple-ai-service. Keep retrieval and orchestration outside the existing booking, admin, registration, and identity services. This keeps AI isolated, easier to evolve, and simpler to secure.

1. Why AI is useful in the current app

- It can answer temple-specific questions using your own data instead of generic internet answers.
- It can guide devotees through bookings, donations, and membership flows without making them search multiple screens.
- It can help admins draft notifications, summarize temple activity, and review member actions faster.
- It can become a temple-scoped assistant instead of a generic chatbot, which is where RAG matters most.

What you should build first

Start with a temple assistant in the Chat tab. Let it answer questions from temple notices, FAQs, timings, donation instructions, booking rules, and user-specific status APIs.

2. Recommended technology stack

Layer	Recommendation
Mobile UI	Existing React Native + Expo app. Add a dedicated Chat tab with card-based responses and action buttons.
AI orchestration	New Python FastAPI microservice named temple-ai-service.
Generation model	OpenAI GPT-4.1 for the first version. Good tool calling, broad context handling, and stable instruction following.
Embedding model	text-embedding-3-small first. Low cost and strong enough for temple content retrieval.
Higher retrieval option	text-embedding-3-large later if precision becomes a problem for multilingual or subtle semantic search.
Vector store	Postgres with pgvector in a separate temple_ai database.
API style	OpenAI Responses API for tool calling and agent loops.
Streaming	Optional in phase 2. Start with non-streaming responses, then add streaming later if chat UX needs it.

3. Model strategy

- Use embeddings to retrieve relevant data. Embeddings are not the answer model; they are the search representation.
- Use GPT-4.1 to read retrieved context, decide whether tools are needed, and generate the final answer.

- Use structured tool calls when the assistant must read live user data such as donation status, booking status, or membership status.

Question	Answer
What retrieves data?	Embeddings + vector similarity search in pgvector.
What writes the answer?	GPT-4.1 through the Responses API.
What checks live app state?	Tool calls from temple-ai-service to existing microservices.
When to consider GPT-5 later?	When you want more advanced multi-step reasoning, better planning, or more complex admin workflows.

4. Target architecture

- Frontend never talks directly to OpenAI.
- Frontend talks to temple-ai-service.
- temple-ai-service retrieves temple knowledge from pgvector and calls OpenAI.
- temple-ai-service also calls existing microservices when live state is needed.

Component	Role
Temple App	Collects user question, shows answers, shows action cards, and launches flows such as Book or Donate.
temple-ai-service	Owns RAG pipeline, tool calling, prompt construction, response formatting, and conversation policy.
temple_ai database	Stores source documents, chunks, metadata, embeddings, conversation references, and retrieval audit data.
temple-admin-service	Provides temple notices, wall of fame, timings, policies, payment instructions, slot rules, and future admin tools.
temple-registration-service	Provides booking status, donation status, membership status, and user transaction history.
temple-identity-service	Provides user profile, role, approved primary temple, and security checks.
OpenAI API	Provides embeddings and response generation with tool orchestration.

5. Where the RAG pipeline should be created

Create the RAG pipeline inside a new microservice named temple-ai-service. Do not mix it into the current gateway or booking services. Retrieval, prompt building, AI safety rules, and tool orchestration should stay isolated there.

1. Ingest temple content from existing services or admin uploads.
2. Normalize the text and split it into chunks.
3. Generate embeddings for each chunk.
4. Store chunks and vectors with metadata such as temple_id, content_type, language, source_id, and visibility.
5. At question time, embed the user query and search only within the selected temple scope.
6. Assemble top results into the prompt.
7. If the query needs live data, call tools before final response generation.
8. Return a final answer plus UI actions such as Open Booking, Open Donate, or Draft Notification.

6. Detailed RAG pipeline

Stage	What it does
Ingestion	Read content from temple notices, wall of fame, FAQs, rules, temple profile, donation instructions, and booking instructions.
Chunking	Break larger documents into small, meaningful passages. Target semantic chunks, not fixed arbitrary blocks only.
Embedding	Use text-embedding-3-small to convert each chunk into a vector.
Storage	Store raw chunk text, metadata, and vector in temple_ai Postgres with pgvector.
Retrieval	On each user query, embed the query and retrieve top-k chunks filtered by temple_id and visibility.
Optional reranking	Not needed in v1. Add later only if answer quality is weak.
Prompt assembly	Build a system prompt, user query, retrieved context, and any live tool results into one final model request.
Answer generation	Use GPT-4.1 to answer only from retrieved and tool-backed context.
Citations	Return source snippets or source titles to reduce hallucination and improve trust.

Important design rule

Temple knowledge retrieval must always be filtered by temple_id. A devotee of one temple must not receive

content from another temple unless you deliberately build a cross-temple mode later.

7. Agentic UI in this app

Agentic UI means the assistant does not only return text. It can also return structured cards, buttons, confirmations, and next actions. In your app, this is more useful than plain chatbot text.

Example input	Agentic UI output
Devotee question	What is the Shantidhara process?
Assistant response	Short answer + Book Shantidhara button + Open temple rules link.
Devotee question	I want to donate 501 rupees.
Assistant response	Donation summary card + Open Donate flow button.
Admin question	Draft a Paryushan information notice.
Assistant response	Draft notification card + Edit + Publish action.
Admin question	Show pending payment proofs.
Assistant response	Review queue card with open actions.

8. Best AI features to build in the current app

- Temple Help Assistant for devotees: timings, booking rules, donation instructions, FAQs, notices.
- Personal Temple Assistant: membership status, booking status, donation proof status, what to do next.
- Admin Drafting Assistant: draft Information and Wall of Fame posts.
- Admin Operations Assistant: summarize pending requests, payment proofs, and recent notifications.
- Future multilingual assistant: Hindi, English, and local language support if temple communities require it.

9. Suggested database design for temple-ai-service

Table	Purpose
source_documents	One row per temple document or content source. Fields: document_id, temple_id, source_type, source_ref, title, raw_text, language, visibility, version, created_at, updated_at.
source_chunks	Chunk-level rows. Fields: chunk_id, document_id, temple_id, chunk_text, chunk_index, token_estimate, embedding_vector, metadata_json.

chat_sessions	Session tracking. Fields: session_id, user_id, temple_id, role, created_at, updated_at.
chat_messages	User and assistant messages. Fields: message_id, session_id, sender_type, content, tool_calls_json, citations_json, created_at.
retrieval_logs	Audit table for which chunks were used. Helpful for debugging and quality review.
tool_audit_logs	Audit of which service tools were called by the assistant and why.

10. Tool layer design

RAG should answer knowledge questions, but user-specific and real-time answers must come from tools. This prevents stale answers and reduces hallucination.

Tool	Owned by
get_membership_status(user_id, temple_id)	registration service
get_booking_status(user_id, temple_id)	registration service
get_donation_status(user_id, temple_id)	registration service
get_available_shantidhara_slots(temple_id, date)	admin service
get_temple_profile(temple_id)	admin service
draft_information_notice(context)	AI only, no external tool needed
publish_information_notice(temple_id, payload)	admin service, admin-only
publish_wall_of_fame(temple_id, payload)	admin service, admin-only

11. Security and governance

- Always enforce role checks before tool execution.
- Always enforce temple_id filtering on retrieval.
- Never let the model call publish tools without explicit admin permissions and confirmation UI.
- Do not include raw secrets, tokens, internal IDs, or payment screenshots in retrieval context.
- Keep an audit trail of tool calls and retrieved chunks for production support.
- Add prompt rules that the assistant must not invent temple policies when source data is missing.

12. Step-by-step implementation plan

9. Create temple-ai-service as a new FastAPI microservice.
10. Create a new temple_ai Postgres database and enable pgvector.

11. Add ingestion jobs for temple notices, wall of fame, payment instructions, FAQs, and booking rules.
12. Implement chunking and embeddings with text-embedding-3-small.
13. Implement retrieval API filtered by temple_id and role visibility.
14. Integrate OpenAI Responses API with GPT-4.1 and structured tool calling.
15. Build Chat tab UI with message list, assistant responses, and action cards.
16. Add devotee tools first: membership status, booking status, donation status.
17. Add admin drafting tools second: draft notification and draft wall of fame.
18. Add citations, logging, and quality review dashboards.
19. Run evaluation on real temple questions before wider rollout.

13. Learning roadmap

Use this order if the team wants to learn and implement step by step without getting overwhelmed.

20. Learn embeddings and vector search fundamentals.
21. Learn the OpenAI Responses API and tool calling.
22. Build simple retrieval first before any agent behavior.
23. Add temple-scoped metadata filters.
24. Add one live tool at a time.
25. Only then build richer agentic UI actions.

Learning topic	Reference
OpenAI Responses API	https://platform.openai.com/docs/api-reference/responses/retrieve
OpenAI file search guide	https://platform.openai.com/docs/guides/tools-file-search/
OpenAI GPT-4.1 model page	https://platform.openai.com/docs/models/gpt-4.1
OpenAI embeddings guide	https://platform.openai.com/docs/guides/embeddings/what-are-embeddings
text-embedding-3-small	https://platform.openai.com/docs/models/text-embedding-3-small
text-embedding-3-large	https://platform.openai.com/docs/models/text-embedding-3-large
OpenAI retrieval guide	https://platform.openai.com/docs/guides/retrieval
pgvector project	https://github.com/pgvector/pgvector

14. Terminology glossary

Term	Definition
RAG	Retrieval-Augmented Generation. The model answers using retrieved project or domain data instead of relying

	only on pretraining.
Embedding	A numeric vector representation of text used for semantic similarity search.
Vector store	A database that stores embeddings and supports similarity search.
Chunking	Splitting larger content into smaller passages so retrieval can find the most relevant part.
Metadata filter	A filter such as <code>template_id</code> , language, or role that limits which chunks can be retrieved.
Retrieval	The search step that finds relevant chunks before answer generation.
Reranking	A second pass that reorders retrieved chunks for better relevance.
Grounding	Making the answer rely on retrieved or tool-backed facts instead of unsupported guesses.
Hallucination	When the model states something confidently that is not supported by source data.
Tool calling	Letting the model ask your application to run a defined function or API action.
Agent	A model workflow that can reason, choose tools, and continue toward a goal.
Agentic UI	A user interface where assistant responses include actions, cards, buttons, or workflow steps, not only plain text.
Structured output	A response format with fixed fields that can be safely parsed by the app.
Prompt	The instructions and context given to the model.
Context window	The maximum amount of input text the model can consider in one request.
Ingestion	The pipeline that reads source data and prepares it for retrieval.
Citation	A source reference shown with the answer to improve trust and traceability.
Tenant isolation	Keeping one temple's data separated from another temple's data.

15. Final recommendation

Recommended first release

Build a temple-scoped assistant in a new temple-ai-service using GPT-4.1 for responses and text-embedding-3-small for retrieval. Start with temple knowledge questions plus three live tools: membership status, booking status, and donation status. Add admin drafting help after that.